

VM and Database as a Service (DBaaS). Both Services in Two Different Providers.

1st Marko Georgiev - 222007

Faculty of Computer Science and Engineering
Ss. Cyril and Methodius University in Skopje
Skopje, North Macedonia
marko.georgiev@students.finki.ukim.mk

Abstract—Cloud applications are increasingly deployed across multiple providers to combine managed compute, networking, and database services from different platforms. This paper presents a performance evaluation of a cross-cloud Proof-of-Work Job Processing Service in which Google Cloud Compute Engine virtual machines process CPU-intensive jobs while Azure Database for PostgreSQL is used as the persistent job queue and result store. The system was evaluated under increasing workload intensities from 1 to 1000 submitted requests per second, using a proof-of-work task configured to require approximately two seconds of computation per job. The evaluation measures request-to-completion time, worker computation time, completed throughput, speedup, autoscaling behavior, elasticity, and cost. The results show that raw proof-of-work computation time remains relatively stable, with average worker execution mostly between 1.7 and 2.1 seconds under higher loads. However, end-to-end completion time increases substantially as the submitted workload exceeds processing capacity, reaching approximately 90 seconds on average and around 170 seconds at the 95th percentile near the highest load. Completed throughput increases with load until the system reaches saturation, stabilizing at approximately 5 to 5.5 jobs per second. The Google Cloud Managed Instance Group successfully scales from one to six virtual machines, but further speedup is limited by the 12-vCPU project quota. The benchmark cost remains low, with an estimated total infrastructure cost of USD 0.368. The findings indicate that the architecture provides effective autoscaling and low-cost execution within quota limits, while high-load performance is primarily constrained by finite compute capacity and queueing delays.

Index Terms—Cloud computing, multi-cloud architecture, virtual machines, database as a service, proof-of-work, autoscaling, performance evaluation, PostgreSQL

I. INTRODUCTION

Cloud infrastructure enables applications to be deployed across managed compute, networking, and storage services without requiring direct ownership of physical hardware. In this project, the application layer is hosted on Google Cloud Platform using Compute Engine virtual machines managed through a Managed Instance Group and exposed through a Load Balancer. The data layer is hosted separately on Azure Database for PostgreSQL, which acts as the persistent job queue and results store for proof-of-work tasks. This separation creates a cross-cloud architecture in which the virtual machines receive requests, execute CPU-intensive computation, and coordinate job state through a database service located in a different cloud provider.

Organizations may choose multi-cloud infrastructure to avoid vendor lock-in, improve resilience, use specialized services from different platforms, or distribute workloads according to cost and performance requirements. Google Cloud Platform and Microsoft Azure are among the leading cloud providers, making them suitable platforms for evaluating practical cross-cloud deployment behavior.

This project makes the following contributions:

- Evaluate how a proof-of-work processing service behaves under increasing request rates and autoscaling conditions.
- Measures speedup, execution time, and throughput as the system scales from one to six VM instances.
- Analyzes scalability, elasticity, and cost implications of the deployed cross-cloud architecture.

II. PROBLEM DESCRIPTION

The project implements a Proof-of-Work Job Processing Service deployed across two cloud providers. The application layer runs on Google Cloud Compute Engine, while the persistent database layer is hosted on Azure PostgreSQL. The purpose of the system is to process computationally expensive proof-of-work jobs submitted by clients through an HTTP API.

Each client request may create a new PoW job. A job consists of input data and a difficulty value. The system must find a numeric nonce such that the SHA-256 hash of `input_data + : + nonce` starts with a required number of leading zero characters, whereby the number of zeroes is equal to the difficulty. For example, if the difficulty is 6, the resulting hash must begin with `000000`.

The database is central to the application. When a client submits a request, the API inserts a new row into the Azure PostgreSQL PoW jobs table with status `queued`. This row represents a pending unit of work. Worker processes running on the Google Cloud VM instances continuously look for `queued` jobs in the same table. Once a worker claims a job, it marks it as `running`, performs the proof-of-work computation, and then updates the database row with the discovered nonce, the resulting hash, execution time, completion timestamp, and final status.

III. SYSTEM ARCHITECTURE

A. Cloud Infrastructure

The implementation uses a two-provider architecture. Google Cloud Platform hosts the application layer, while Azure hosts the database layer.

The compute infrastructure is deployed using a Google Compute Engine Managed Instance Group (pow-worker-mig) in the europe-west1-b zone, specifically in Belgium. Instances are created from the pow-worker-template-v5 template and use the e2-standard-2 machine type (2 vCPUs), while autoscaling and autohealing are enabled through the managed instance group.

The application is packaged as a Docker image and stored in Google Artifact Registry. During startup, each VM installs Docker, retrieves the database connection string from Google Secret Manager, pulls the latest image, and starts the containerized FastAPI application. Each container runs two worker processes (`WORKER_COUNT=2`).

Incoming requests are handled by the external Application Load Balancer pow-load-balancer, which forwards traffic to the backend service connected to the managed instance group. Instance health is monitored through the pow-health-check endpoint /health on port 8000.

Persistent storage is provided by Azure Database for PostgreSQL. All proof-of-work jobs, statuses, and results are stored in the database, allowing VM instances to be replaced or scaled without losing application state.

The worker VMs do not have public IP addresses. Outbound connectivity is provided through Cloud NAT (pow-cloud-nat), allowing instances to access Artifact Registry and external services while remaining private.

B. Runtime Topology

The execution flow is shown below:

- 1) A client sends a proof-of-work request to the Google Cloud Load Balancer.
- 2) The load balancer forwards the request to a healthy VM instance.
- 3) The FastAPI application inserts a new job into Azure PostgreSQL.
- 4) Local worker processes poll the database and claim queued jobs.
- 5) The worker performs the proof-of-work computation.
- 6) The result, nonce, and execution information are written back to PostgreSQL.
- 7) The client retrieves the job status or final result through the API.

Because all workers share the same PostgreSQL database, any VM instance can process jobs and serve status requests. It's important to note that the initial goal of the project was to provide evaluation and measurement where the time to consider a request completed would be the time from event 1) to event 7) in the defined runtime topology.

C. Autoscaling Design

The managed instance group is configured with CPU-based autoscaling. The group currently scales between one and six VM instances, with a target CPU utilization of 60%.

Each VM runs two worker processes, increasing the total processing capacity as additional instances are created. New instances are launched from the instance template and become available after completing the startup script and passing the health check.

Autohealing is enabled using the same health check. If an instance becomes unhealthy, the managed instance group automatically replaces it.

D. Technical Limitations

The main implementation limitation was the resource quota available in Google Cloud. The project was restricted to a maximum of 12 vCPUs and a maximum of 8 VM instances. These limits constrained the range of autoscaling experiments that could be performed and prevented testing larger configurations.

Because the selected instances used the e2-standard-2 machine type, the practical maximum deployment size was six VMs. This provided enough capacity to demonstrate autoscaling behavior, but it was not sufficient to keep up with the highest submitted request rates.

E. Database Design

The database layer is implemented using Azure Database for PostgreSQL. The deployed server is named pow-postgres and runs PostgreSQL version 16.13 in the Azure Austria east region. The server uses the Burstable Standard_B1ms configuration, with 1 vCore, 2 GiB of memory, and 32 GiB of provisioned storage.

The database is accessed through the public PostgreSQL endpoint pow-ostgres.postgres.database.azure.com. Access is restricted using Azure firewall rules. The main production access rule allows the Google Cloud NAT public IP address 35.240.124.109, which is used by the private Google Cloud VM instances when connecting outbound to Azure PostgreSQL through their Cloud NAT.

The database contains two main tables. The pow_jobs table stores the state of each proof-of-work request. Each row contains the job identifier, input data, difficulty value, job status, creation and execution timestamps, worker identifier, computed nonce, result hash, execution time, end-to-end time, and any error message. Additional experiment-related columns store the benchmark experiment identifier, target request rate, run label, submission timestamp, and API response timestamp.

IV. WORKLOAD GENERATOR

A custom duration-based workload generator was used to evaluate the Proof-of-Work service under increasing request rates. The workload generator submits requests asynchronously to the FastAPI service through the load balancer. Each target request rate is maintained for a fixed duration.

In this experiment, the duration per rate was set to one second, meaning that the number of submitted jobs for each level was equal to the configured target requests per second. For example, the 100 RPS test submitted 100 jobs, while the 1000 RPS test submitted 1000 jobs. The tested request rates were: 1, 10, 50, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000 requests per second (rps).

This duration was chosen, due to the hard limitation of 12 allowed vCPUs, durations longer than 1 severely skew the results making them much harder to interpret.

Each submitted job used proof-of-work difficulty 5. The difficulty is configured so that each job takes ≈ 2 seconds of **computation time**, not accounting for queue wait time. This value was selected for the workload generator runs in order to create CPU-bound processing while keeping the benchmark duration practical across multiple request-rate levels. The generator also recorded that each VM instance was configured with two local worker processes, matching the deployed container configuration. Lastly, the generator also sampled the current number of VM instances in the managed instance group `pow-worker-mig`, allowing the benchmark results to be correlated with the autoscaling behavior of the Google Cloud deployment.

V. RESULTS AND EVALUATION

A. Execution time

Execution time is defined as the end-to-end time from the moment a client request is submitted to the public load balancer until the corresponding proof-of-work job is marked as completed in Azure PostgreSQL. This metric captures the complete user-observed processing delay, including HTTP request handling, job insertion, queue waiting time, worker execution, and the final database update.

Figure 1 shows the request-to-completion time as the number of submitted requests increases. The average and median completion times increase significantly at higher loads, reaching approximately 90 seconds near 1000 submitted requests.



Fig. 1. From request to completion time (including queuing).

The p95 latency grows even more sharply, reaching approximately 170 seconds. This indicates that the dominant delay at

high request volumes is not the hash computation itself, but the queueing delay caused by more jobs waiting in PostgreSQL before workers can process them. Figure 2 shows the worker computation time without queueing. Unlike the end-to-end plot, the average computation time remains relatively stable, mostly between 1.7 and 2.1 seconds for higher request counts.

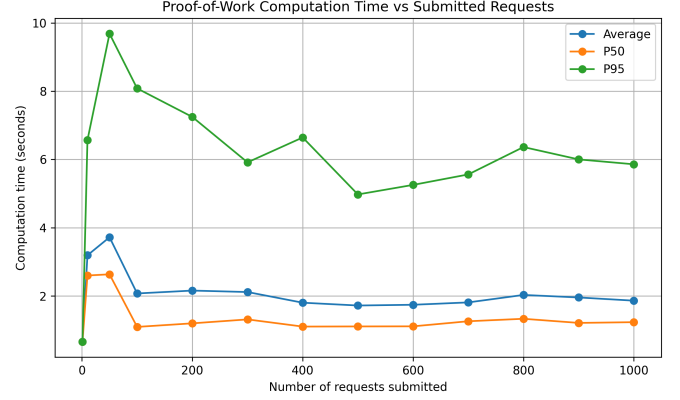


Fig. 2. PoW completing mapped against job claimed by VM.

The median is also stable, usually close to 1.1–1.3 seconds. The p95 values are higher, around 5–6 seconds for larger runs, because proof-of-work has variable runtime: some jobs find a valid nonce quickly, while others require more hashing attempts.

The contrast between the two plots shows that the system’s computational cost remains mostly bounded, while the end-to-end execution time grows due to accumulated queueing. This behavior is expected for a CPU-bound workload with a **finite number of worker processes**. As the arrival rate increases beyond the processing capacity of the active VM workers, jobs spend more time in the database queue before execution begins.

The final evaluation for execution time is **POOR**, as user-visible completion time becomes dominated by queueing delays under high load, resulting in poor overall execution performance.

B. Throughput

Throughput measures the number of proof-of-work jobs completed per second. It is not equal to the number of worker processes. In this system, each VM runs two local workers, but the measured throughput depends on the time required to complete jobs, queue delays, autoscaling behavior, and database updates. Therefore, throughput is computed as the number of completed jobs divided by the elapsed time between request submission and database completion.

The main throughput metric used in this experiment is completed throughput including queueing. It is calculated from the completed jobs over the full request-to-database-finish window, so it includes the time jobs spend waiting in Azure PostgreSQL before being claimed by a worker. This makes it the most representative measure of user-visible system throughput.

Figure 3 compares completed throughput, ideal worker-capacity throughput, and submitted throughput as the number of submitted requests increases.

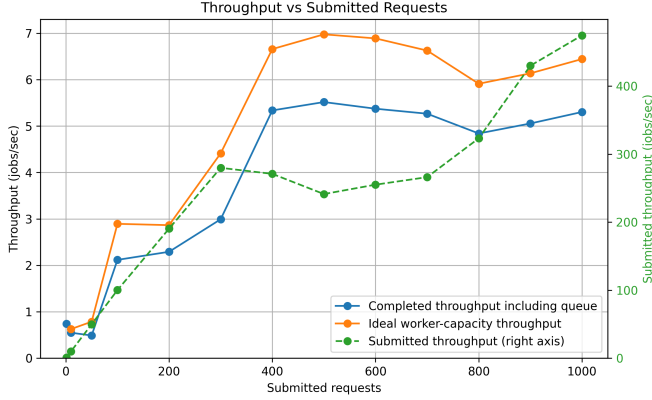


Fig. 3. Completed throughput, ideal worker-capacity throughput, and submitted throughput.

Completed throughput initially rises as the workload increases, reaching approximately 5–5.5 jobs/s once enough jobs are available to keep the workers busy. After this point, throughput remains mostly flat, even as submitted load continues to increase. This indicates that the system reaches its processing capacity and additional requests accumulate in the queue instead of increasing completed work per second. This number additionally makes sense as we choose a difficulty that would, on average require 2 seconds of work to complete, considering there are 12 workers when the systems scales until it reaches it's limit, means 12 workers (vCPUs) / ≈ 2 seconds per PoW job ≈ 6 jobs per second. Where, requests taking longer than 2 seconds, account for the dip shown as the number of requests reach 800.

Figure 4 shows a better depiction of completed throughput (the lefthand Y-axis) against the target request rate. Although the target request rate (the righthand Y-axis) increases up to 1000 requests/s, completed throughput remains bounded at roughly 5 jobs/s. This confirms that the workload generator is able to submit requests faster than the system can process proof-of-work jobs. The system therefore becomes compute-bound and queue-bound at higher request rates.

The final evaluation for throughput is **GOOD-POOR**, as the system achieves throughput close to its expected processing capacity before saturating, indicating good performance within available resources.

C. Speedup

Speedup measures the relative improvement in completed throughput compared with the 1 request/s baseline. It is calculated as the completed throughput at each load level divided by the completed throughput observed at 1 request/s. This metric is used to evaluate how effectively the system converts additional submitted load and autoscaled worker capacity into completed proof-of-work jobs. In Figure 5 we can clearly see

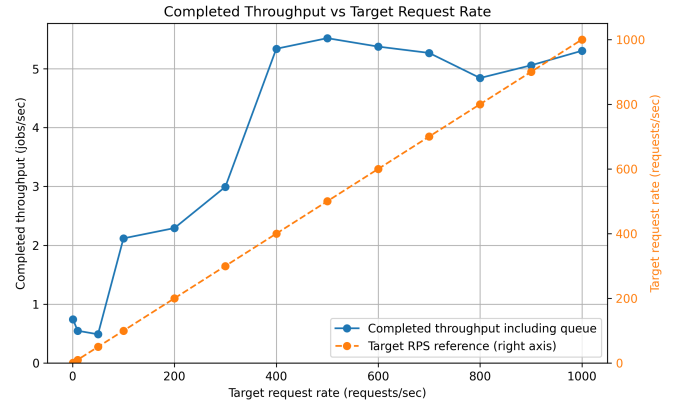


Fig. 4. Completed throughput against the target request rate.

the technical limitation in action, as the actual speedup from the Google Cloud Managed Instance Group could not keep up cannot compete with the ideal one, due to the 12 vCPUs limitation.

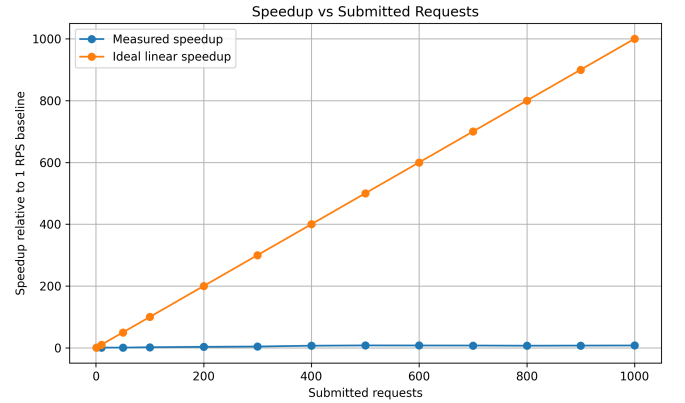


Fig. 5. Measured speedup compared with ideal linear speedup.

The final evaluation for speedup is **POOR**, as performance improvement is limited and stops increasing once the fixed 12-vCPU worker capacity is reached.

D. Scalability

Scalability describes how the system behaves as the submitted request rate increases from 1 request/s to 1000 request/s. In this experiment, the system is expected to scale by increasing the number of active Google Cloud VM instances in the managed instance group, where each VM contributes two local worker processes.

The observed managed instance group size in Figure 6 increased as the submitted load increased. The deployment started with a small number of active instances and scaled upward as the request volume grew. The average observed group size reached the configured maximum of six VM instances at around 400 submitted requests and remained at that level for the rest of the experiment. This shows that

the autoscaler reacted to the CPU-intensive proof-of-work workload and increased compute capacity under higher load.

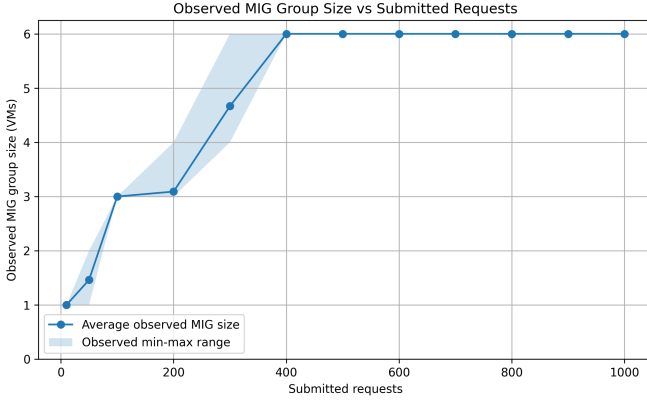


Fig. 6. Group size against requests per second.

The important part is that, as can be seen in Figure 7, CPU drops to almost idle after the benchmark finishes, while the MIG remains at 6 VMs for some time before scaling down.

The final evaluation for scaling is **GOOD**, as the autoscaler successfully increases capacity as demand grows, but scalability is constrained by the six-VM quota.

E. Elasticity

Elasticity describes the ability of the infrastructure to automatically adjust resources in response to workload changes. In this experiment, elasticity is evaluated through the behavior of the Google Cloud Managed Instance Group under increasing and decreasing proof-of-work load.

The autoscaler reacted correctly to the benchmark workload. As CPU utilization increased, the Managed Instance Group scaled out from one to six VMs (Figure 7). The autoscaler recommendation matched the actual group size during scale-out, showing that the autoscaling policy was active and responsive to the CPU-bound workload. The CPU signal repeatedly exceeded the target utilization, which triggered the increase in VM count.

The monitoring data indicates that the dominant limitation was compute capacity rather than network or disk saturation. During the benchmark window from 17:35 until 18:11 (Figure 7), CPU utilization repeatedly reaches high levels, often around 70–90%, while the Managed Instance Group scales upward from 1 VM to the maximum of 6 VMs. Since each VM is e2-standard-2, the system reaches the quota limit of 6 VMs $\times$ 2 vCPUs = 12 vCPUs. After the MIG reaches 6 VMs, it cannot scale further, so additional load cannot be absorbed by adding workers. This explains why throughput stagnates and end-to-end completion time increases.

The scale-in behavior also shows the time required for the system to return to its minimum capacity. In the observed run, the benchmark workload ended at approximately 18:11, after which CPU utilization dropped close to idle. However, the Managed Instance Group remained at six VM instances until

TABLE I
ESTIMATED COST OF THE BENCHMARK RUN

Component	Cost (USD)
Azure PostgreSQL Flexible Server	0.028
Google Compute Engine VMs	0.306
Cloud NAT gateway	0.006
Cloud NAT external IP address	0.005
External LB forwarding rule	0.023
Artifact Registry	0.000
Total	0.368

approximately 18:28, when it returned to one VM. This means that the system took roughly 17 minutes to scale back down after the workload ended.

The final evaluation for elasticity is **GOOD**, as the system responds well to increasing load through scale-out, although scale-in is slower and leaves excess resources running after the workload ends.

F. Cost Analysis

The database component was deployed using Azure Database for PostgreSQL Flexible Server. The selected configuration was the Burstable Standard_B1ms compute size, providing 1 vCore, 2 GiB of memory, and 32 GiB of Premium SSD storage. This tier is intended for development and testing workloads, which matches the experimental nature of this project. Azure estimated the monthly database cost as follows: compute cost was USD 16.06 per month for the Standard_B1ms instance, and storage cost was USD 4.42 per month for 32 GiB of provisioned storage. The estimated total monthly database cost was therefore USD 20.48.

For a time-based interpretation, the nominal hourly database cost can be approximated as $\frac{20.48}{730} \approx 0.028$ USD/hour. The database was used for approximately one hour during the experiment. Based on the estimated hourly cost, the nominal cost of operating the PostgreSQL database for the benchmark was therefore approximately $0.028 \times 1 \approx 0.028$ USD. This corresponds to roughly 2.8 US cents for the duration of the experiment.

The Google Cloud component was estimated using the observed Managed Instance Group size during the experiment. The relevant experiment window was from 17:35 to 18:31, including the time required for the system to scale back down to one VM. This corresponds to 56 minutes of measured infrastructure activity. During that interval 274 VM minutes were used or $\frac{274}{60} = 4.567$ VM-hours. Since the Managed Instance Group used e2-standard-2 instances, and the estimated price for this machine type in europe-west1, Belgium, is approximately USD 0.067 per VM-hour, the nominal VM cost for the experiment was $4.567 \times 0.067 \approx 0.306$ USD.

Additional Google Cloud services were also part of the deployment. These include Cloud NAT, the external load balancer, and Artifact Registry. Their estimated costs are shown together with the database and VM costs in Table I. The table only includes measurable fixed infrastructure costs for the experiment window.

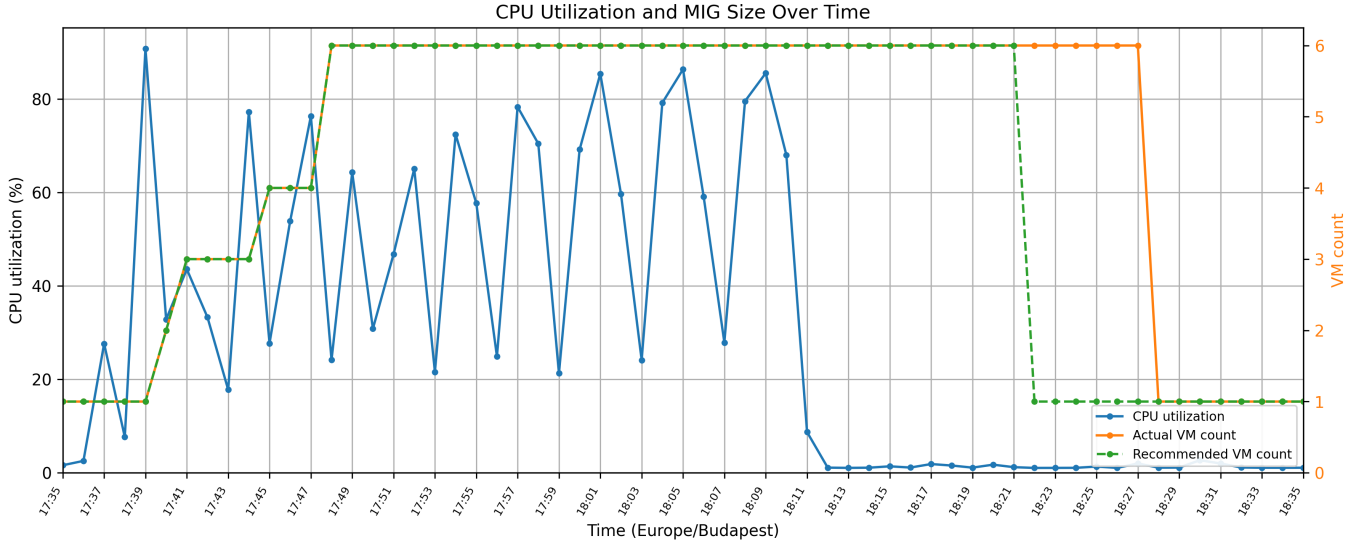


Fig. 7. CPU utilization against number of MIG instances.

The final cost analysis is evaluated as **EXCELLENT**, due to infrastructure costs remain below one US dollar for the benchmark, demonstrating excellent cost efficiency.

Variable bandwidth charges are excluded because the exact billable cross-cloud data transfer volume was not available.

VI. CONCLUSION

This project evaluated a cross-cloud Proof-of-Work Job Processing Service using Google Cloud virtual machines and Azure Database for PostgreSQL. The results show that worker computation time remained stable at around 1.7–2.1 seconds, but end-to-end completion time increased sharply under high load because jobs accumulated in the database queue. Completed throughput saturated at approximately 5–5.5 jobs per second once the Managed Instance Group reached its six-VM, 12-vCPU limit. The system demonstrated good autoscaling and elasticity within the available quota, but speedup was limited by fixed compute capacity. The benchmark cost was low, with an estimated total of USD 0.368. Future improvements should focus on increasing VM quota, using larger instances, tuning worker count per VM, and monitoring queue length directly so that scaling decisions better reflect user-visible delay.